

DSA STUDY BLUEPRINT SERIES

32.3 Sum Brute Better Optimized

Finding Unique Triplets summing up to Zero

Section 05 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Sorting & Pointers:** Sort array, fix index i , and use two-pointer sweeps on remainder.
- **De-duplication:** Skip identical neighbors to prevent generating duplicate triplets.

Memory & Execution Blueprint

3-Sum Pin Index:

nums[i] (Pinned) • nums[l] (Left sweep) → ←
nums[r] (Right sweep)

C++ Implementation Reference

Standard code layout and syntax

```
vector<int> threeSum(vector<int>& nums) {  
    sort(nums.begin(), nums.end());  
    vector<int> ans;  
    for (int i = 0; i < nums.size(); i++) {  
        if (i > 0 && nums[i] == nums[i-1]) continue;  
        int l = i + 1, r = nums.size() - 1;  
        while (l < r) {  
            int sum = nums[i] + nums[l] + nums[r];  
            if (sum == 0) {  
                ans.push_back({nums[i], nums[l++], nums[r--]});  
                while (l < r && nums[l] == nums[l-1]) l++;  
            } else if (sum < 0) l++;  
            else r--;  
        }  
    }  
    return ans;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N^2)$
Space Complexity	$O(1)$ (excluding output)

Key Practices & Gotchas

- **Important:** Sorting the array allows effective duplicates pruning and two-pointer sweeps.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace check complement lookups in a hash map for `Two Sum` on `[2, 7, 11]` (target = 9):

Element Scan	Complement: <code>target - x</code>	Hash Map Search	Map Insertion
2	$9 - 2 = 7$	Not found	Insert 2 mapping index 0
7	$9 - 7 = 2$	Found 2 (at index 0!)	Return indices [0, 1]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Prefix Sum Math:** Resolve sub-array queries using math transformations. Complement hash lookups count occurrences instantly.
- **Hash Collisions:** Handled via separate chaining (lists link) or open addressing (linear index probing).
- **Related LeetCode Problems:** #560 Subarray Sum Equals K, #1 Two Sum, #49 Group Anagrams.